

Data Analysis Algorithm

Week-4

Task

Dhusyanth. R

CH.SC.U4CSE24161

1.) BST Balancing using Rotation motion:

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int key;
6      struct Node *left, *right;
7      int height;
8  };
9
10 int max(int a, int b) {
11     return (a > b) ? a : b;
12 }
13
14 int getHeight(struct Node *n) {
15     return (n == NULL) ? 0 : n->height;
16 }
17
18 struct Node* newNode(int key) {
19     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
20     node->key = key;
21     node->left = node->right = NULL;
22     node->height = 1;
23     return node;
24 }
```

```

26 struct Node* rightRotate(struct Node *y) {
27     struct Node *x = y->left;
28     struct Node *T2 = x->right;
29
30     x->right = y;
31     y->left = T2;
32
33     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
34     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
35
36     return x;
37 }
38
39 struct Node* leftRotate(struct Node *x) {
40     struct Node *y = x->right;
41     struct Node *T2 = y->left;
42
43     y->left = x;
44     x->right = T2;
45
46     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
47     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
48
49     return y;
50 }
51
52 struct Node* balanceTree(struct Node* node) {
53     if (node == NULL)
54         return NULL;
55
56     node->left = balanceTree(node->left);
57     node->right = balanceTree(node->right);
58
59     node->height = 1 + max(getHeight(node->left), getHeight(node->right));
60     int balance = getHeight(node->left) - getHeight(node->right);
61
62     // Left Left Case
63     if (balance > 1 && getHeight(node->left->left) >= getHeight(node->left->right))
64         return rightRotate(node);
65
66     // Left Right Case
67     if (balance > 1 && getHeight(node->left->left) < getHeight(node->left->right)) {
68         node->left = leftRotate(node->left);
69         return rightRotate(node);
70     }
71
72     // Right Right Case
73     if (balance < -1 && getHeight(node->right->right) >= getHeight(node->right->left))
74         return leftRotate(node);

```

```

76 // Right Left Case
77 if (balance < -1 && getHeight(node->right->right) < getHeight(node->right->left)) {
78     node->right = rightRotate(node->right);
79     return leftRotate(node);
80 }
81
82 return node;
83 }
84
85 void printGivenLevel(struct Node* root, int level) {
86     if (root == NULL)
87         return;
88
89     if (level == 1)
90         printf("%d ", root->key);
91     else if (level > 1) {
92         printGivenLevel(root->left, level - 1);
93         printGivenLevel(root->right, level - 1);
94     }
95 }
96

```

```

97 void printLevelOrder(struct Node* root) {
98     int h = getHeight(root);
99     for (int i = 1; i <= h; i++) {
100         printGivenLevel(root, i);
101         printf("| ");
102     }
103 }
104
105 int main() {
106     printf("CH.SC.U4CSE24161  \n");
107     public int __cdecl printf (const char * __restrict__ _Format, ...)
108     struct Node *root = newNode(157);
109     root->left = newNode(110);
110     root->left->right = newNode(147);
111     root->left->right->left = newNode(122);
112     root->left->right->right = newNode(149);
113     root->left->right->right->right = newNode(151);
114     root->left->right->left->left = newNode(111);
115     root->left->right->left->right = newNode(141);
116     root->left->right->left->left->right = newNode(112);
117     root->left->right->left->right->left = newNode(123);
118     root->left->right->left->right->left->right = newNode(133);
119     root->left->right->left->left->right->left = newNode(117);
120
121     root = balanceTree(root);
122     root = balanceTree(root);
123
124     printf("AVL Level Order: ");
125     printLevelOrder(root);
126     printf("\n");
127
128     return 0;
}

```

Output:

```

CH.SC.U4CSE24161
AVL Level Order: 122 | 110 157 | 117 147 | 111 112 133 149 | 123 141 151 |
-----
Process exited after 0.0361 seconds with return value 0
Press any key to continue . . .

```

Time Complexity: $O(n)$

Justification: The `balanceTree` function is called once on the root, but it recursively visits every node in the tree exactly once to calculate heights and perform rotations. Since the code manually builds a tree with n nodes and then runs a single post-order traversal to balance it, the time taken is proportional to the number of nodes.

Note: While standard AVL insertions are $O(\log n)$, your specific implementation balances an existing unbalanced tree in a single pass, making it linear for the total number of nodes.

Space Complexity: $O(\log n)$

Justification: The space complexity is determined by the maximum depth of the function call stack during recursion. Since an AVL tree is strictly balanced, its height h is guaranteed to be $O(\log n)$. Even though you start with an unbalanced tree, the recursive depth of `balanceTree` or `printLevelOrder` will not exceed the initial height of the tree. No significant auxiliary data structures (like arrays or queues) are used, only the memory for the nodes themselves and the recursion stack.

2.) Red Black Tree:

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  enum Color { RED, BLACK };
5
6  struct Node {
7      int data;
8      enum Color color;
9      struct Node *left, *right, *parent;
10 };
11
12 struct Node* newNode(int data) {
13     struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
14     temp->data = data;
15     temp->left = temp->right = temp->parent = NULL;
16     temp->color = RED;
17     return temp;
18 }
19
20 void rotateLeft(struct Node** root, struct Node* x) {
21     struct Node* y = x->right;
22     x->right = y->left;
23
24     if (y->left != NULL)
25         y->left->parent = x;
```

```

27     y->parent = x->parent;
28
29     if (x->parent == NULL)
30         *root = y;
31     else if (x == x->parent->left)
32         x->parent->left = y;
33     else
34         x->parent->right = y;
35
36     y->left = x;
37     x->parent = y;
38 }
39
40 void rotateRight(struct Node** root, struct Node* y) {
41     struct Node* x = y->left;
42     y->left = x->right;
43
44     if (x->right != NULL)
45         x->right->parent = y;
46
47     x->parent = y->parent;
48

```



```

49     if (y->parent == NULL)
50         *root = x;
51     else if (y == y->parent->left)
52         y->parent->left = x;
53     else
54         y->parent->right = x;
55
56     x->right = y;
57     y->parent = x;
58 }
59
60 void fixViolation(struct Node** root, struct Node* z) {
61     while (z != *root && z->parent->color == RED) {
62
63         if (z->parent == z->parent->parent->left) {
64
65             struct Node* y = z->parent->parent->right;
66
67             if (y != NULL && y->color == RED) {
68                 z->parent->color = BLACK;
69                 y->color = BLACK;
70                 z->parent->parent->color = RED;
71                 z = z->parent->parent;

```

71
72
73 ☐
74
75
76 ☐
77
78
79
80
81 ☐
82
83
84
85
86
87 ☐
88
89
90
91
92
93 ☐
94
95

```
        z = z->parent->parent;
    } else {
        if (z == z->parent->right) {
            z = z->parent;
            rotateLeft(root, z);
        }

        z->parent->color = BLACK;
        z->parent->parent->color = RED;
        rotateRight(root, z->parent->parent);
    }

} else {

    struct Node* y = z->parent->parent->left;

    if (y != NULL && y->color == RED) {
        z->parent->color = BLACK;
        y->color = BLACK;
        z->parent->parent->color = RED;
        z = z->parent->parent;
    } else {
        if (z == z->parent->left) {
            z = z->parent;
            rotateRight(root, z);
```

```

96         }
97
98         z->parent->color = BLACK;
99         z->parent->parent->color = RED;
100         rotateLeft(root, z->parent->parent);
101     }
102 }
103 }
104
105 (*root)->color = BLACK;
106 }
107
108 int height(struct Node* node) {
109     if (node == NULL)
110         return 0;
111
112     int l = height(node->left);
113     int r = height(node->right);
114
115     return (l > r ? l : r) + 1;
116 }
117
118 void printGivenLevel(struct Node* root, int level) {
119     if (root == NULL)
120         return;
121
122     if (level == 1)
123         printf("%d ", root->data);
124     else {
125         printGivenLevel(root->left, level - 1);
126         printGivenLevel(root->right, level - 1);
127     }
128 }

```

```

129
130 void printLevelOrder(struct Node* root) {
131     int h = height(root);
132
133     for (int i = 1; i <= h; i++) {
134         printGivenLevel(root, i);
135         printf("| ");
136     }
137 }
138
139 int main() {
140
141     printf("CH.SC.U4CSE24161\n");
142
143     struct Node* root = newNode(157);
144     root->color = BLACK;
145
146     struct Node* n110 = newNode(110);
147     root->left = n110; n110->parent = root;
148
149     struct Node* n147 = newNode(147);
150     n110->right = n147; n147->parent = n110;
151
152     struct Node* n122 = newNode(122);
153     n147->left = n122; n122->parent = n147;
154
155     struct Node* n149 = newNode(149);
156     n147->right = n149; n149->parent = n147;
157
158     struct Node* n111 = newNode(111);
159     n122->left = n111; n111->parent = n122;
160
161     struct Node* n141 = newNode(141);
162     n122->right = n141; n141->parent = n122;
163
164     struct Node* n151 = newNode(151);
165     n149->right = n151; n151->parent = n149;

```

```

166
167     struct Node* n112 = newNode(112);
168     n111->right = n112; n112->parent = n111;
169
170     struct Node* n123 = newNode(123);
171     n141->left = n123; n123->parent = n141;
172
173     struct Node* n117 = newNode(117);
174     n112->left = n117; n117->parent = n112;
175
176     struct Node* n133 = newNode(133);
177     n123->right = n133; n133->parent = n123;
178
179     fixViolation(&root, n147);
180     fixViolation(&root, n111);
181     fixViolation(&root, n141);
182     fixViolation(&root, n151);
183     fixViolation(&root, n117);
184     fixViolation(&root, n133);
185
186     printf("Red-Black Level Order: ");
187     printLevelOrder(root);
188     printf("\n");
189
190     return 0;
191

```

Output:

```

CH.SC.U4CSE24161
Red-Black Level Order: 122 | 111 147 | 110 112 133 151 | 117 123 141 149 157 |
-----
Process exited after 0.04356 seconds with return value 0
Press any key to continue . . .

```

Time Complexity: $O(n \log n)$

Justification: While a single insertion and balance (fixViolation) in a Red-Black Tree takes $O(\log n)$

time, this code manually links nodes and then calls `fixViolation` multiple times in a sequence.

- **Balancing:** Each call to `fixViolation` takes $O(\log n)$ as it may traverse up to the root.
- **Printing:** The `printLevelOrder` function uses a nested loop and recursion that visits nodes level-by-level. For a balanced tree, this specific level-order implementation (without a queue) takes $O(n)$ time.
- **Overall:** For n elements, if you were inserting them normally, it would be $O(n \log n)$.

Space Complexity: $O(n)$

Justification: The program requires memory to store each element as a `struct Node`, which includes pointers for left, right, parent, and the color attribute. This auxiliary memory is directly proportional to the number of nodes n in the tree. Furthermore, the recursive functions for height and level-printing use the function call stack, which requires $O(\log n)$ space for a balanced Red-Black Tree.